

# VETO: Video Efficient Token Optimization for Vision-Language Models

Gueter Josmy Faure<sup>1</sup> Hao Ping Wang<sup>1</sup> Min-Hung Chen<sup>2</sup> Winston H. Hsu<sup>1</sup>

<sup>1</sup>National Taiwan University <sup>2</sup>NVIDIA

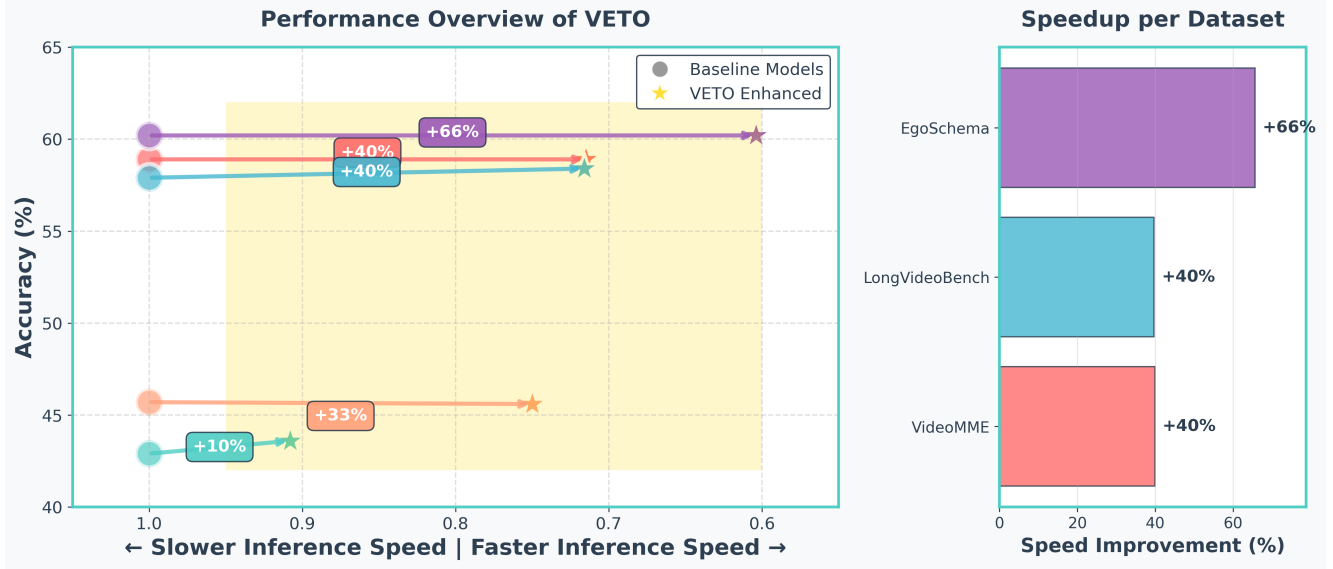


Figure 1. VETO performance overview. Our dual-axis compression places VETO in the efficiency–accuracy sweet spot: it reduces inference latency across model scales and benchmarks while preserving or improving task accuracy.

## Abstract

Processing long videos with Vision-Language Models (VLMs) is bottlenecked by the quadratic cost of visual tokens, making long-form inference prohibitively expensive. While single-axis compression methods mitigate this, they hit a hard efficiency floor because they treat spatial and temporal redundancy independently. We present **VETO** (Video Efficient Token Optimization), a training-optional plug-in that eliminates this bottleneck through dual-axis compression: (i) an **intra-frame** compressor that merges semantically similar tokens within each frame via optimal-transport inspired matching, and (ii) an **inter-frame** compressor that identifies and merges temporally redundant frames. The key design insight is hierarchical ordering: by first compressing spatial dimensions, VETO drastically reduces the cost of subsequent global temporal matching—bypassing the efficiency wall of single-axis approaches, with an advantage that grows with modern fully-fused attention infras-

tructure. Empirically, VETO achieves up to **45%** faster inference (e.g., on LLaVA-OneVision-7B) while preserving or improving accuracy. Under extreme token starvation (10% budget), VETO outperforms VFlowOpt [32] (54.9%), VisionZip [31] (52.6%), and FastV [3] (47.9%) with **55.7%** accuracy. We demonstrate universal applicability across LLaVA-OneVision, InternVL-2.5, and LongVA, with zero-shot accuracy preserved or improved in all cases.

## 1. Introduction

Vision-Language Models (VLMs) have transformed multimodal reasoning, enabling systems to answer questions, summarize scenes, and follow instructions grounded in visual input [1, 6, 16]. Yet scaling these systems to long-form video remains a critical bottleneck: attention costs grow quadratically with the total number of visual tokens. For a video with  $T$  frames and  $N$  tokens per frame, the complexity is  $O((TN)^2)$ , meaning multi-minute videos quickly

produce tens of thousands of tokens, rendering inference prohibitively expensive.

Existing solutions trade off accessibility and performance. Training-based methods require substantial compute. Training-free plug-ins (e.g., token pruning, merging, or frame selection) [2, 3, 7] are immediately deployable, yet they overwhelmingly treat spatial and temporal redundancy independently. This single-axis paradigm harbors a critical flaw: relying solely on inter-frame compression requires computing similarity over *fully uncompressed* spatial tokens, inflating latency. Single-axis methods therefore hit a hard efficiency floor, preventing optimal speed-accuracy trade-offs.

We introduce **VETO** (Video Efficient Token Optimization), a training-optional plug-in that resolves this through a *dual-axis* strategy. VETO applies (i) an *Intra-Frame Compressor* that merges semantically similar tokens within each frame using optimal-transport-inspired assignment, followed by (ii) an *Inter-Frame Compressor* that globally identifies and merges temporally redundant frames. By executing spatial compression *before* temporal matching, VETO slashes the feature dimensions required for global similarity calculations, mitigating the latency bottleneck of single-axis methods. This reduces effective attention complexity to  $O(TN^2/k + T^2C)$  (with  $C \ll N$ ), enabling substantial runtime savings.

A critical property of VETO’s design is that its advantage over single-axis baselines is *hardware-agnostic*—and in fact *grows* on modern, optimized attention infrastructure. Fully-fused attention kernels effectively absorb per-frame spatial reductions from intra-only methods, but cannot absorb VETO’s total-sequence-length reduction. This means VETO’s relative speedup margin over single-axis baselines increases on modern deployments, making it an increasingly compelling choice as inference stacks continue to be optimized.

**System contribution.** Beyond the algorithmic design, VETO is immediately deployable as a drop-in module requiring no custom CUDA kernels or architectural modifications. It operates as a pure tensor transformation—applied between the visual encoder and the LLM—and is fully compatible with standard attention implementations including FlashAttention. The end-to-end latency measurements reported in this paper (visual encoder + token compression + LLM inference) constitute a realistic systems-level evaluation of deployment cost.

We validate VETO across three VLM families: LLaVA-OneVision [12] (0.5B and 7B), InternVL-2.5 [5] (1B), and LongVA [34] (7B). VETO consistently reduces end-to-end latency while preserving or improving accuracy. Under a strict 10% token budget, VETO achieves 55.7%, outperforming VFlowOpt (54.9%) and VisionZip (52.6%). In the fully supervised setting (Section 4.3), VETO additionally

achieves state-of-the-art on long-form narrative benchmarks and competitive procedural action recognition.

Our contributions are:

- **Hardware-agnostic dual-axis compression:** A hierarchical intra-then-inter ordering that resolves the efficiency floor of single-axis methods, with a growing advantage on modern optimized attention infrastructure.
- **Universal, training-optional plug-in:** VETO integrates into diverse frozen VLMs (LLaVA, InternVL, LongVA) with no architectural changes, no custom kernels, and no retraining required.
- **Extreme-budget robustness:** VETO establishes a new state-of-the-art under severely constrained token budgets (10% retention), outperforming all evaluated single-axis baselines.

## 2. Related Work

**Vision-Language Models.** VLMs bridge visual perception and natural language processing [6, 13, 16, 19]. Contrastive pre-training (CLIP [19]) aligned image and text at scale; instruction-tuned successors such as LLaVA [16], InstructBLIP [6], and Qwen-VL [1] leveraged large language models as reasoning engines. For video, VideoChat [14], Video-LLaMA [33], and Video-ChatGPT [18] extend image-based VLMs to the temporal domain. All share the same scalability challenge: processing long videos generates tens of thousands of visual tokens, creating quadratic attention complexity that limits practical deployment.

**Efficiency plug-ins for VLMs.** Token-reduction methods include pruning [20], merging (ToMe [2]), and adaptive selection [4]. FastV [3] prunes visual tokens based on attention statistics; VisionZip [31] compresses the visual context for image VLMs; VFlowOpt [32] guides token pruning with visual information flow. ST-LLM [17] introduces spatial-temporal token selection for video. LLaVA-PruMerge [22] combines pruning and merging for image VLMs. VideoLLM-MoD [30] applies mixture-of-depths to video. HERMES [7] provides training-free hierarchical memory for long videos. Unlike these works, which operate along a single spatial or temporal axis, VETO introduces dual-axis compression with a strict hierarchical ordering—exploiting the complementary nature of spatial and temporal redundancy to achieve latency reductions that grow with the efficiency of the underlying attention kernel.

## 3. VETO: Video Efficient Token Optimization

VETO is a training-optional plug-in that addresses the computational bottleneck in video understanding through dual-axis compression: spatial compression within frames (*Intra-Frame Compressor*) followed by temporal compression across the sequence (*Inter-Frame Compressor*). This hierarchical ordering ensures that the expensive temporal

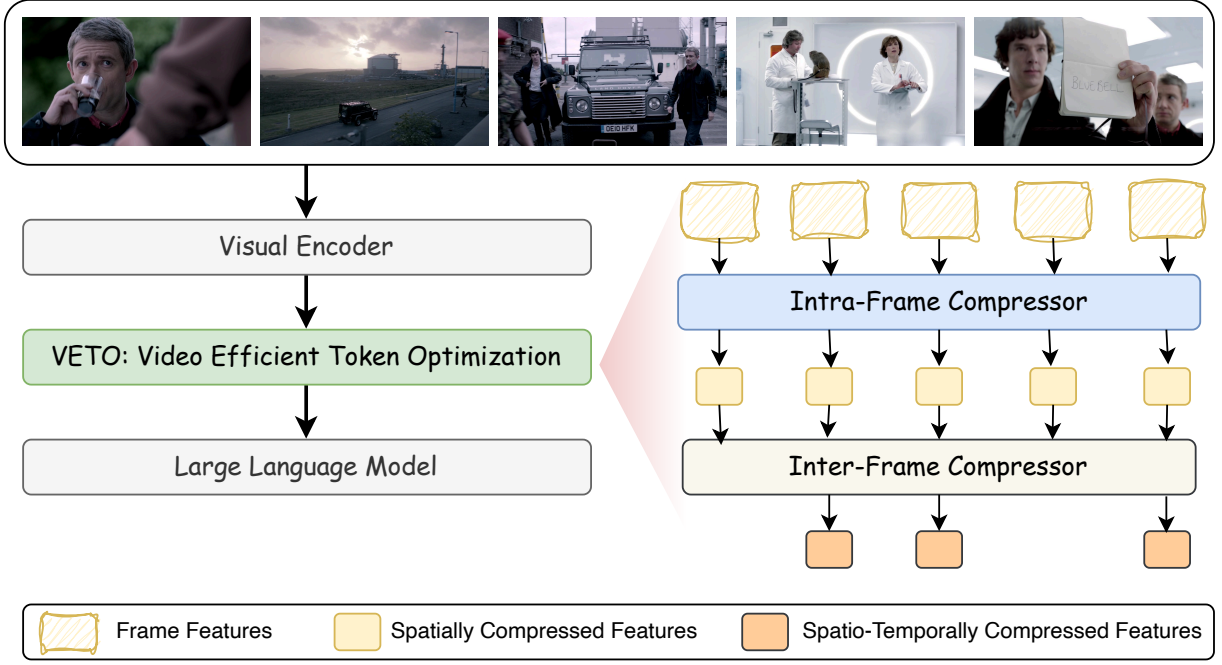


Figure 2. VETO architecture. Visual encoder features are first compressed by the *Intra-Frame Compressor* (semantic token merging within each frame) and then by the *Inter-Frame Compressor* (frame-level merging). The compressed representation maintains feature dimensionality and is fed to the frozen LLM without architectural changes.

similarity matching operates on an already-reduced spatial dimension, unlocking substantial latency reductions.

### 3.1. Motivation: The Complexity Bottleneck

Standard VLM attention scales as  $\mathcal{O}((T \cdot N)^2)$ . Single-axis temporal methods reduce  $T$  but must first compute a global similarity matrix across *fully uncompressed* spatial tokens, introducing a strict hardware-level latency floor that prevents optimal speed-ups. VETO bypasses this floor through hierarchical ordering. Intra-frame spatial compression first reduces the spatial dimension to  $N/k$  at a cost of only  $\mathcal{O}(N^2/k)$  per frame. The subsequent inter-frame temporal compression then operates on this shrunken feature space, requiring only  $\mathcal{O}(T^2 \cdot C)$  for global similarity assessment (typically  $C \ll N$ ). The effective attention complexity becomes  $\mathcal{O}(T \cdot N^2/k + T^2 \cdot C)$ —fundamentally breaking the single-axis latency floor.

### 3.2. Problem Formulation

Given a video  $\mathcal{V} = \{I_1, \dots, I_T\}$ , the visual encoder produces a memory bank  $\mathbf{M} \in \mathbb{R}^{B \times T \times N \times C}$ . We seek compression functions  $f_{\text{intra}} : \mathbb{R}^{B \times T \times N \times C} \rightarrow \mathbb{R}^{B \times T \times N' \times C}$  and  $f_{\text{inter}} : \mathbb{R}^{B \times T \times N' \times C} \rightarrow \mathbb{R}^{B \times T' \times N' \times C}$  with  $N' < N$  and  $T' < T$ , minimizing information loss:

$$\mathcal{L} = \mathbb{E}_{\mathcal{V}} [\mathcal{D}(\mathcal{F}(\mathbf{M}), \mathcal{F}(f_{\text{inter}}(f_{\text{intra}}(\mathbf{M}))))] \quad (1)$$

where  $\mathcal{F}$  is downstream task performance and  $\mathcal{D}$  measures semantic preservation.

### 3.3. Intra-Frame Compressor

The Intra-Frame Compressor reduces spatial redundancy within individual frames by merging semantically similar tokens. Let  $\mathbf{X}_t \in \mathbb{R}^{N \times C}$  be the token features for frame  $t$ , extracted from the frozen visual encoder prior to any LLM projection.

We employ a  $k$ -way splitting strategy inspired by optimal transport. Tokens are partitioned into “destination” anchors and “source” tokens:

$$\mathbf{X}_{\text{src}} = \mathbf{X}_t[\text{idx} \not\equiv 0 \pmod{k}, :] \in \mathbb{R}^{N_{\text{src}} \times C} \quad (2)$$

$$\mathbf{X}_{\text{dst}} = \mathbf{X}_t[\text{idx} \equiv 0 \pmod{k}, :] \in \mathbb{R}^{N_{\text{dst}} \times C} \quad (3)$$

where  $N_{\text{dst}} = \lfloor N/k \rfloor$  and  $N_{\text{src}} = N - N_{\text{dst}}$ . We then compute the cosine similarity matrix between source and destination tokens:

$$\mathbf{S} = \frac{\mathbf{X}_{\text{src}} \mathbf{X}_{\text{dst}}^T}{\|\mathbf{X}_{\text{src}}\|_2 \|\mathbf{X}_{\text{dst}}\|_2} \quad (4)$$

and assign each source to its most similar destination:

$$\text{idx}_i = \arg \max_j \mathbf{S}_{i,j}, \quad i \in [1, N_{\text{src}}] \quad (5)$$

Merged tokens are formed by size-weighted averaging:

$$\mathbf{X}'_{t,j} = \frac{\sum_{i: \text{idx}_i=j} \mathbf{X}_{\text{src},i} + \mathbf{X}_{\text{dst},j}}{|\{i : \text{idx}_i = j\}| + 1} \quad (6)$$

This reduces the spatial token count from  $N$  to approximately  $N/k$ , outputting the spatially compressed bank  $\mathbf{M}' \in \mathbb{R}^{B \times T \times N' \times C}$ .

**Rationale for uniform partitioning.** The modulo- $k$  partition is spatially *uniform by design*: every spatial region contributes exactly one anchor, guaranteeing complete spatial coverage. Content-adaptive anchor selection concentrates anchors in salient regions, leaving peripheral areas without valid destinations—leading to precisely the spatial collapse one might expect from aggressive compression. In VETO, content-adaptiveness is reserved for the *merging* step (Eq. 5): each source token is matched to its cosine-nearest anchor, achieving semantic grouping. Adding content-adaptive anchor scoring would require an additional  $\mathcal{O}(N^2)$  pass, negating the efficiency gain. Empirically, Table 4 confirms accuracy is stable across  $k \in \{2, 3, 4, 5\}$ ; significant spatial collapse would manifest as monotonic degradation, which is not observed.

**Positional information.** VETO operates on feature vectors already output by the frozen ViT encoder, which has baked absolute positional information into each token prior to compression. The downstream LLM receives compressed tokens as a flat visual prefix with no independent spatial position IDs—the standard convention for LLaVA-style models [16]. For models with RoPE-based temporal embeddings (e.g., LongVA [34]), the LLM assigns sequential indices to the compressed visual sequence after merging. Since intra-frame merging groups spatially adjacent tokens (modulo- $k$  groups span at most  $k$  consecutive positions), positional drift is bounded and empirically harmless, as confirmed by the ablation in Table 4.

### 3.4. Inter-Frame Compressor

The Inter-Frame Compressor addresses temporal redundancy by identifying and merging similar frames. It operates *after* intra-frame compression, computing temporal similarity on the already-reduced spatial dimension  $N'$ .

#### 3.4.1. Global Similarity Assessment

Given  $\mathbf{M}' \in \mathbb{R}^{B \times T \times N' \times C}$ , frame representations are obtained by spatial mean-pooling:

$$\mathbf{F}_t = \frac{1}{N'} \sum_{n=1}^{N'} \mathbf{M}'_{:,t,n,:} \in \mathbb{R}^{B \times C} \quad (7)$$

The pairwise cosine similarity matrix is:

$$\mathbf{G}_{i,j} = \frac{\mathbf{F}_i \cdot \mathbf{F}_j}{\|\mathbf{F}_i\|_2 \|\mathbf{F}_j\|_2}, \quad i, j \in [1, T] \quad (8)$$

---

### Algorithm 1 Inter-Frame Compression

---

**Require:** Memory bank  $\mathbf{M}' \in \mathbb{R}^{B \times T \times N' \times C}$ , target size  $T'$ , merges per step  $m$

**Ensure:** Compressed memory bank  $\mathbf{M}'' \in \mathbb{R}^{B \times T' \times N' \times C}$

```

1: Initialize active mask  $\mathbf{A} \in \{0, 1\}^{B \times T}$ 
2: count  $\leftarrow T$ 
3: while count  $> T'$  do
4:    $\mathbf{F}_t \leftarrow \text{mean}(\mathbf{M}'_{:,t,:,:})$ ,  $\mathbf{F}_t \leftarrow \mathbf{F}_t / \|\mathbf{F}_t\|_2$ 
5:    $\mathbf{G} \leftarrow \mathbf{F} \mathbf{F}^T \odot \mathbf{W}$ 
6:    $\{(i_k, j_k)\}_{k=1}^m \leftarrow \text{top-}m(\mathbf{G})$ 
7:   for each pair  $(i_k, j_k)$  do
8:      $\mathbf{M}''_{j_k} \leftarrow \text{weighted\_avg}(\mathbf{M}'_{i_k}, \mathbf{M}'_{j_k})$ 
9:      $\mathbf{A}_{i_k} \leftarrow 0$ 
10:  end for
11:  count  $\leftarrow \text{count} - m$ 
12: end while
13: return  $\mathbf{M}''$ 
```

---

A greedy algorithm then selects the top- $m$  most similar non-overlapping pairs for merging, applying a valid-pair mask  $\mathbf{W}_{i,j} = 1$  iff  $i < j$  and both frames are active. Selected frame pairs are merged by compression-size weighted averaging:

$$\mathbf{M}''_{j_k} = \frac{\mathbf{M}'_{i_k} \cdot s_{i_k} + \mathbf{M}'_{j_k} \cdot s_{j_k}}{s_{i_k} + s_{j_k}} \quad (9)$$

This iterates until the target frame count  $T'$  is reached. The number of iterations equals the reduction in frame count (e.g.,  $64 \rightarrow 48$  frames requires 16 iterations), not the video length, keeping the overhead small.

## 4. Experiments

### 4.1. Implementation Details

All plug-in experiments inject VETO after the visual encoder and before the LLM, preserving per-token feature dimensionality. Latency is measured end-to-end (visual encoder + token compression + LLM inference) at batch size 1. Primary latency results use **H100+FlashAttention-2**; for InternVL-2.5-1B and LLaVA-OneVision-0.5B, V100 latency is reported directly (full H100 comparisons in supplementary material). Vanilla baseline scores and VETO evaluations share a unified codebase. Accuracy for FastV [3], VisionZip [31], and VFlowOpt [32] is taken from the unified re-evaluation in [32]; their hardware latency was not benchmarked.

Table 2 summarizes the hyperparameter settings used across all benchmarks. For zero-shot evaluations, VETO is injected without any fine-tuning, using a fixed intra-frame factor  $k = 2$  (retaining 50% of spatial tokens per frame) and an inter-frame retention ratio  $G_{\text{ratio}} = 0.75$  for video benchmarks. For fully supervised tasks (Section 4.3), only a lightweight linear adapter is optimized; the visual encoder and LLM remain frozen throughout training.

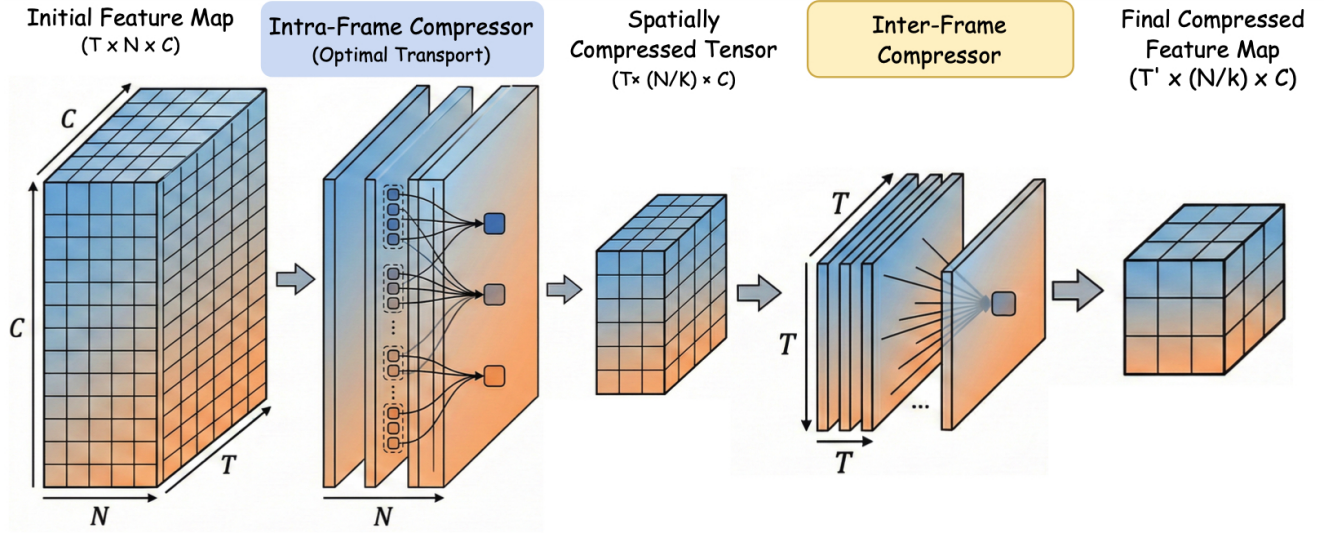


Figure 3. **The VETO dual-axis pipeline and dimensionality reduction.** **First (Intra-Frame):** Optimal-transport-inspired matching reduces spatial token count from  $N$  to  $N/k$ , preserving salient semantics while shrinking the feature tensor. **Second (Inter-Frame):** Global frame-level redundancy is eliminated. Because the temporal similarity matrix is computed on the already spatially compressed dimension ( $N/k$ ), computational overhead is minimized.

Table 1. Performance across benchmarks. All latency normalized to Vanilla = 1.00 ( $\downarrow$  is better).  $^\dagger$  Latency on H100+FlashAttention-2.  $^\ddagger$  Latency on V100; full H100 comparison in supplementary.  $\star$  VideoMME accuracy from unified re-evaluation [32]: FastV 55.9%, VisionZip 57.5%, VFlowOpt 58.0%—all degrade from the 58.9% Vanilla baseline.

| (a) VideoMME & LongVideoBench  |                    |             |             |                |             |
|--------------------------------|--------------------|-------------|-------------|----------------|-------------|
| Model                          | Method             | VideoMME    |             | LongVideoBench |             |
|                                |                    | Acc.        | Lat.        | Acc.           | Lat.        |
| LLaVA-OneV. (7B) $^\dagger$    | Vanilla            | <b>58.9</b> | 1.00        | 57.9           | 1.00        |
|                                | ToMe               | 58.3        | 0.96        | 58.3           | 0.98        |
|                                | HERMES             | 58.4        | 0.93        | 57.1           | 0.95        |
|                                | <b>VETO (Ours)</b> | <b>58.9</b> | <b>0.85</b> | <b>58.4</b>    | <b>0.86</b> |
| LLaVA-OneV. (0.5B) $^\ddagger$ | Vanilla            | 42.9        | 1.00        | 46.0           | 1.00        |
|                                | ToMe               | <b>43.8</b> | 1.36        | <b>46.4</b>    | 0.60        |
|                                | HERMES             | 43.5        | 0.92        | 44.8           | 0.61        |
|                                | <b>VETO (Ours)</b> | 43.6        | <b>0.91</b> | 45.9           | <b>0.55</b> |
| LongVA (7B) $^\dagger$         | Vanilla            | <b>54.0</b> | 1.00        | 52.9           | 1.00        |
|                                | ToMe               | 53.3        | 0.93        | 52.7           | 0.95        |
|                                | HERMES             | 53.1        | 0.95        | 52.9           | 0.95        |
|                                | <b>VETO (Ours)</b> | <b>53.5</b> | <b>0.90</b> | <b>53.0</b>    | <b>0.86</b> |
| InternVL-2.5 (1B) $^\ddagger$  | Vanilla            | <b>49.2</b> | 1.00        | <b>45.7</b>    | 1.00        |
|                                | <b>VETO (Ours)</b> | 48.1        | <b>0.81</b> | 45.6           | <b>0.75</b> |

(b) EgoSchema (V100 lat.)

| Method                      | Acc.        | Lat.        |
|-----------------------------|-------------|-------------|
| <i>LLaVA-OneVision (7B)</i> |             |             |
| Vanilla                     | 60.2        | 1.00        |
| ToMe                        | <b>60.5</b> | 0.78        |
| HERMES                      | 60.0        | 0.82        |
| <b>VETO</b>                 | 60.2        | <b>0.60</b> |

(c) Image Benchmarks (LLaVA-OV 7B, V100 lat.)

| Method      | Acc.            | Lat.        |
|-------------|-----------------|-------------|
| <i>MME</i>  |                 |             |
| Vanilla     | <b>418/1580</b> | 1.00        |
| <b>VETO</b> | 418/1571        | <b>0.88</b> |
| <i>MMU</i>  |                 |             |
| Vanilla     | <b>48.8</b>     | 1.00        |
| <b>VETO</b> | 48.3            | <b>0.73</b> |

## 4.2. Integration with Vision-Language Models

**Video understanding benchmarks.** Table 1a reports accuracy and normalized latency across four VLM families. H100+FlashAttention-2 latency is reported for LLaVA-OV-7B and LongVA-7B ( $^\dagger$ ); V100 latency for InternVL-2.5-1B and LLaVA-OV-0.5B ( $^\ddagger$ ), with full H100 results in supplementary material.

mentary material.

On **LLaVA-OneVision-7B**, VETO achieves **58.9%** VideoMME accuracy (matching Vanilla) with a **15% latency reduction** ( $0.85\times$ ) on H100. ToMe and HERMES, operating on the *identical* LLaVA-OneVision-7B backbone, manage only 4% and 7% reductions respectively. VETO’s margin is  **$3.75\times$  larger than ToMe’s**, not a backbone ad-

Table 2. Hyperparameter settings.  $k$ : intra-frame factor (retains  $1/k$  spatial tokens per frame);  $G$ : frame retention fraction. **Abbrevs:** VME = VideoMME; LVB = LongVideoBench; ES = EgoSchema; MC = MovieChat-1K; BF = BreakFast.

| (a) Zero-Shot |        |     |      | (b) Fully Supervised |        |     |      |     |
|---------------|--------|-----|------|----------------------|--------|-----|------|-----|
|               | Frames | $k$ | $G$  |                      | Frames | $k$ | $G$  | Ep. |
| VME           | 64     | 2   | 0.75 | BF                   | 100    | 2   | 0.32 | 20  |
| LVB           | 64     | 2   | 0.75 | COIN                 | 100    | 2   | 0.32 | 20  |
| ES            | 64     | 2   | 0.75 |                      |        |     |      |     |
| MC            | 40     | 2   | 0.50 |                      |        |     |      |     |

vantage, but a structural one: total-sequence-length reduction cannot be absorbed by fused attention kernels, whereas per-frame spatial reductions can. As reported in [32], FastV (55.9%), VisionZip (57.5%), and VFlowOpt (58.0%) all degrade accuracy below the 58.9% Vanilla baseline.

#### Finding 1

Same backbone, same hardware, same benchmark: VETO cuts latency by **15%** while ToMe manages 4% and HERMES 7%. This **3.75 $\times$  larger margin** comes with *zero accuracy penalty*.

On **LongVA-7B**, VETO achieves the best accuracy *and* best latency on both benchmarks simultaneously: 53.5%/0.90 $\times$  on VideoMME and 53.0%/0.86 $\times$  on LongVideoBench, outperforming ToMe (53.3%/0.93 $\times$ ) and HERMES (53.1%/0.95 $\times$ ). A different architecture, the same story: cross-architecture robustness confirmed.

For **InternVL-2.5-1B**, ToMe and HERMES lack implementation support; we compare against the Vanilla baseline. VETO achieves competitive accuracy (48.1% vs. 49.2% VideoMME) with a **19% V100 latency reduction**.

**Long-term reasoning and image benchmarks.** On EgoSchema (Table 1b), VETO achieves a **40% latency reduction** while matching vanilla accuracy (60.2%)—nearly doubling the efficiency gain over ToMe (22%) and HERMES (18%), on the identical LLaVA-OneVision-7B backbone. For image benchmarks (Table 1c), VETO maintains identical MME perception scores (418/1580) with a 12% speedup, and achieves 48.3% on MMMU (vs. 48.8%) with a **27% speedup**, confirming that spatial redundancy is exploited effectively even for static images.

### 4.3. Fully Supervised Results

The plug-in experiments establish VETO’s deployment value. But do the same compression principles hold—or amplify—when training is introduced? We evaluate a fully-supervised variant where the visual encoder and LLM remain *strictly frozen*; only a lightweight linear adapter is optimized (20 epochs; hyperparameters in Table 2b).

Table 3. (a) MovieChat-1K long-form narrative understanding (all baselines 7B, zero-shot). (b) Procedural action recognition on BreakFast (BF) and COIN (Fully-supervised). † Methods using the identical InstructBLIP (with vicuna-7B) backbone as VETO.

| (a) MovieChat-1K        |             | (b) Procedural Action Recognition |             |             |
|-------------------------|-------------|-----------------------------------|-------------|-------------|
| Model                   | Acc.        | Method                            | BF          | COIN        |
| Video-LLaMA [33]        | 51.7        | VIS4mer [9]                       | 88.2        | 88.4        |
| VideoChat [14]          | 57.8        | Trans4mer [10]                    | 90.3        | 89.2        |
| VideoChatGPT [18]       | 47.6        | S5 [27]                           | 90.7        | 90.8        |
| MovieChat [24]          | 62.3        | MA-LMM <sup>†</sup> [8]           | 93.0        | 93.2        |
| Long-LLaVA [29]         | 69.6        | VTM [11]                          | 91.3        | 88.6        |
| TimeChat [21]           | 73.8        | HERMES <sup>†</sup> [7]           | <b>95.2</b> | <b>93.5</b> |
| HERMES <sup>†</sup> [7] | 78.6        |                                   |             |             |
| <b>VETO (Ours)</b>      | <b>80.6</b> | <b>VETO (Ours)</b>                | <u>94.4</u> | <u>92.7</u> |

**Long-form narrative understanding.** Table 3a shows results on MovieChat-1K. VETO achieves **80.6%** accuracy—a new state-of-the-art and a **2.0%** improvement over HERMES (†), which uses the *identical* InstructBLIP (vicuna-7B) backbone. This is not a model-size artifact: by eliminating redundant tokens, the frozen LLM concentrates representational capacity on semantically critical events, improving coherence across long temporal horizons.

#### Finding 3

New SotA on MovieChat-1K: **VETO 80.6%** vs. HERMES 78.6% (same backbone). This shows compression does not just preserve understanding; it enhances it.

**Procedural action recognition.** On BreakFast and COIN (Table 3b), VETO achieves 94.4% and 92.7% respectively, competitive with dedicated temporal architectures and ranking second overall. The strong performance across both datasets, despite the compression-focused design, validates that hierarchical token merging preserves the fine-grained temporal structure necessary for procedural understanding.

### 4.4. Ablation Studies

We conduct ablation studies using LLaVA-OneVision-7B on VideoMME, LongVideoBench, and MovieChat-1K.

**Token merging strategy.** Table 4a uses Intra-first (VETO) as the reference (1.00) and measures how all other orderings fare against it. Every alternative is slower: Intra-only (1.10–1.14 $\times$ ), Inter-only (1.16 $\times$ ), and even Inter-first (1.07–1.09 $\times$ )—the seemingly symmetric counterpart—incur extra overhead because their similarity computations run on fully uncompressed feature spaces. Intra-first is uniquely efficient: by shrinking spatial dimensions *before* temporal matching, the inter-frame similarity step operates on an already-reduced representation, cutting its cost dramatically. It also achieves the best accuracy on all

Table 4. Ablations (LLaVA-OV 7B, V100 lat.). (a) Token merging strategy, revealing the latency floor of single-axis methods. (b) Intra-frame compression ratios. (c) Inter-frame compression ratios. (d) Token starvation (VideoMME) vs. single-axis baselines.

(a) Merging Strategy & Latency (LLaVA-OneVision 7B)

| Strategy                  | VideoMME    |             | LongVideoBench |             | MovieChat-1K |
|---------------------------|-------------|-------------|----------------|-------------|--------------|
|                           | Acc.        | Lat.        | Acc.           | Lat.        |              |
| Intra-only                | 58.4        | 1.10        | 58.3           | 1.14        | 80.98        |
| Inter-only                | 58.4        | 1.16        | 58.2           | 1.16        | 80.59        |
| Inter-first               | 58.4        | 1.07        | 58.0           | 1.09        | 80.78        |
| <b>Intra-first (VETO)</b> | <b>58.9</b> | <b>1.00</b> | <b>58.4</b>    | <b>1.00</b> | <b>81.18</b> |

(b) Intra-Frame Ratio

| Ratio | VideoMME    | MovieChat-1K |
|-------|-------------|--------------|
| 1/2   | <b>58.9</b> | <b>80.78</b> |
| 1/3   | 58.4        | 80.98        |
| 1/4   | 58.4        | 80.78        |
| 1/5   | 59.0        | 80.98        |

(c) Inter-Frame Ratio

| Ratio | VideoMME    | MovieChat-1K |
|-------|-------------|--------------|
| 0.75  | <b>58.9</b> | <b>80.78</b> |
| 0.50  | 57.9        | 79.80        |
| 0.25  | 56.1        | 79.61        |
| 0.125 | 52.1        | 77.65        |

(d) Token Starvation (VideoMME)

| Method      | 25%         | 10%         |
|-------------|-------------|-------------|
| FastV       | 55.9        | 47.9        |
| VisionZip   | 57.5        | 52.6        |
| VFlowOpt    | 58.0        | 54.9        |
| <b>VETO</b> | <b>58.3</b> | <b>55.7</b> |

three benchmarks simultaneously.

#### Finding 2

Every other ordering is **7–16% slower** than Intra-first (VETO). **Intra-first achieves both best accuracy and lowest latency.**

**Effect of intra-frame compression ratio.** Table 4b shows remarkable stability: accuracy remains within 0.6% across  $k \in \{2, 3, 4, 5\}$  on VideoMME. This confirms that individual frames contain substantial spatial redundancy, and that the uniform-anchor + cosine-nearest-source matching successfully identifies semantically equivalent tokens without damaging the visual representation.

**Effect of inter-frame compression ratio.** Table 4c reveals a clear asymmetry: temporal compression is far more sensitive than spatial compression. At  $G_{\text{ratio}} = 0.75$ , accuracy is fully preserved; below 0.25, degradation becomes substantial (VideoMME drops from 58.9% to 52.1%). This asymmetry reflects a structural property of video: individual frames contain redundant textures and backgrounds, but temporal transitions encode causally critical events that reasoning tasks cannot recover from if lost.

#### Finding 4

**Spatial is forgiving; temporal is not.** Accuracy degrades  $<0.6\%$  across  $4\times$  spatial reduction (Table b). It drops 6.8% at  $4\times$  temporal reduction (Table c). Recommended:  $k = 2$ ,  $G_{\text{ratio}} \geq 0.75$ .

**Robustness under extreme token starvation.** Table 4d stress-tests all methods at 25% and 10% token retention

budgets. At 10% retention, VETO achieves **55.7%** (the only method above 55%). The dual-axis design degrades gracefully as each compressor operates at a moderate individual ratio rather than forcing one axis to overcompensate.

#### Finding 5

At 10% token budget **VETO 55.7% > VFlowOpt 54.9 > VisionZip 52.6 > FastV 47.9**. Dual-axis compression is uniquely suited to extreme-budget inference.

## 4.5. Qualitative Compression Visualizations

Figure 4 visualizes both compression stages. The inter-frame compressor (left) intelligently retains keyframes that capture scene transitions, discarding redundant temporal information. The intra-frame compressor (right) groups semantically similar background patches while preserving distinct foreground objects—demonstrating that similarity-based merging preserves semantically critical regions that uniform downsampling would discard.

## 5. Discussion

**Spatial vs. temporal redundancy asymmetry.** A recurring theme across our ablations is a fundamental asymmetry between the two compression axes. Spatial (intra-frame) compression is highly robust: accuracy on VideoMME remains within 0.6% across  $k \in \{2, 3, 4, 5\}$ , corresponding to retaining between 20% and 50% of per-frame tokens. Temporal (inter-frame) compression is considerably more sensitive: degradation becomes substantial below  $G_{\text{ratio}} = 0.25$ , and moderate at 0.50. This asymmetry is not accidental—it



Figure 4. Qualitative visualizations of VETO’s dual-axis compression. **Left:** The inter-frame compressor retains scene-transition keyframes and discards redundant frames ( $7 \rightarrow 3$ ). **Right:** The intra-frame compressor; patches colored by merge group. Semantically similar background regions (sky, grass, textures) are merged while distinct foreground objects are preserved.

reflects a structural property of video. Individual frames encode highly redundant spatial information (textures, backgrounds, repeated patterns), whereas temporal transitions encode causally important events and state changes that are critical for reasoning tasks. VETO’s design exploits this asymmetry by applying aggressive spatial compression first and conservative temporal compression second, leading to the recommended defaults  $k = 2$ ,  $G_{\text{ratio}} \geq 0.75$ .

**Why the advantage grows on modern hardware.** Fully-fused attention kernels such as FlashAttention-2 reduce the constant factor of the quadratic attention cost and are especially effective at absorbing the per-frame spatial reductions that intra-only methods provide. However, they cannot absorb a reduction in the total sequence length, which is precisely what VETO’s inter-frame stage delivers. Consequently, as attention infrastructure continues to improve, VETO’s margin over single-axis baselines widens rather than shrinks. This makes VETO an increasingly compelling choice as inference stacks are optimized, unlike methods whose advantage is architecture-specific.

**Deployment guidelines.** For practitioners, we recommend the following defaults across deployment contexts: (1) *Latency-critical applications* (real-time video QA):  $k = 2$ ,  $G_{\text{ratio}} = 0.75$  gives 15% latency reduction on H100+FA2 with no accuracy loss. (2) *Budget-constrained edge devices*:  $k = 4$ ,  $G_{\text{ratio}} = 0.50$  is stable spatially but incurs modest temporal accuracy loss—acceptable for scene-level tasks. (3) *Extreme resource constraints* (10% token budget): VETO outperforms all single-axis baselines (55.7% vs. 54.9% for VFlowOpt) because dual-axis compression allows each compressor to operate at a moderate individual ratio rather than forcing one axis to overcompensate.

## 6. Conclusion

We present VETO, a training-optional dual-axis token compression plug-in for video VLMs. As a plug-in, VETO

yields consistent latency reductions across model scales ( $0.85\text{--}0.86\times$  on LLaVA-OneVision-7B;  $0.55\times$  on LLaVA-OneVision-0.5B;  $0.86\text{--}0.90\times$  on LongVA-7B). Its advantage over single-axis baselines grows on modern optimized attention infrastructure, confirming the hardware-agnostic nature of the dual-axis design principle. Under extreme token starvation (10% budget), VETO establishes a new state-of-the-art, outperforming all single-axis methods by a significant margin. In the fully supervised setting (Section 4.3), VETO additionally achieves competitive procedural recognition (Breakfast: 94.4%, COIN: 92.7%).

**Broader impacts.** By reducing token counts by up to 50% spatially and 25% temporally, VETO directly reduces FLOPs and energy consumption during inference, contributing to the goal of sustainable VLM deployment on consumer-grade hardware.

**Limitations.** All plug-in experiments are conducted on models up to 7B parameters. At larger scales, attention patterns, token redundancy profiles, and hardware bottlenecks may differ; empirical validation beyond 7B is left to future work. Additionally, the intra-frame compressor inherits no text-query signal: compression is purely content-based, not query-aware. A query-conditioned extension would add an  $\mathcal{O}(N^2)$  anchor-scoring pass but could improve performance on tasks requiring precise spatial localization.

## References

- [1] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond, 2023. 1, 2
- [2] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster. *arXiv preprint arXiv:2210.09461*, 2022. 2
- [3] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang

- Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision*, pages 19–35. Springer, 2024. 1, 2, 4
- [4] Shixing Chen, Chun-Hao Liu, Xiang Hao, Xiaohan Nie, Maxim Arap, and Raffay Hamid. Movies2scenes: Using movie metadata to learn scene representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6535–6544, 2023. 2
- [5] Zhe Chen, Weiyun Wang, Yue Cao, Yangzhou Liu, Zhangwei Gao, Erfei Cui, Jinguo Zhu, Shenglong Ye, Hao Tian, Zhaoyang Liu, et al. Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *arXiv preprint arXiv:2412.05271*, 2024. 2
- [6] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. Instructblip: Towards general-purpose vision-language models with instruction tuning, 2023. 1, 2
- [7] Gueter Josmy Faure, Jia-Fong Yeh, Min-Hung Chen, Hung-Ting Su, Shang-Hong Lai, and Winston H Hsu. Hermes: temporal-coherent long-form understanding with episodes and semantics. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 22911–22921, 2025. 2, 6
- [8] Bo He, Hengduo Li, Young Kyun Jang, Menglin Jia, Xuefei Cao, Ashish Shah, Abhinav Shrivastava, and Ser-Nam Lim. Ma-Imm: Memory-augmented large multimodal model for long-term video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13504–13514, 2024. 6
- [9] Md Mohaiminul Islam and Gedas Bertasius. Long movie clip classification with state-space video models. In *European Conference on Computer Vision*, pages 87–104. Springer, 2022. 6
- [10] Md Mohaiminul Islam, Mahmudul Hasan, Kishan Shamsundar Athrey, Tony Braskich, and Gedas Bertasius. Efficient movie scene detection using state-space transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18749–18758, 2023. 6
- [11] Seon-Ho Lee, Jue Wang, Zhikang Zhang, David Fan, and Xinyu Li. Video token merging for long video understanding. *Advances in Neural Information Processing Systems*, 37:13851–13871, 2024. 6
- [12] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, et al. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024. 2
- [13] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*, pages 19730–19742. PMLR, 2023. 2
- [14] KunChang Li, Yanan He, Yi Wang, Yizhuo Li, Wenhui Wang, Ping Luo, Yali Wang, Limin Wang, and Yu Qiao. Videochat: Chat-centric video understanding. *arXiv preprint arXiv:2305.06355*, 2023. 2, 6
- [15] Ji Lin, Chuhan Gao, and Song Han. Tsm: Temporal shift module for efficient video understanding. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 7083–7093, 2019. 12
- [16] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023. 1, 2, 4
- [17] Ruyang Liu, Chen Li, Haoran Tang, Yixiao Ge, Ying Shan, and Ge Li. St-llm: Large language models are effective temporal learners. In *European Conference on Computer Vision*, pages 1–18. Springer, 2024. 2
- [18] Muhammad Maaz, Hanoona Rasheed, Salman Khan, and Fahad Shahbaz Khan. Video-chatgpt: Towards detailed video understanding via large vision and language models. *arXiv preprint arXiv:2306.05424*, 2023. 2, 6
- [19] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021. 2
- [20] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. Dynamicvit: Efficient vision transformers with dynamic token sparsification. *Advances in neural information processing systems*, 34:13937–13949, 2021. 2
- [21] Shuhuai Ren, Linli Yao, Shicheng Li, Xu Sun, and Lu Hou. Timechat: A time-sensitive multimodal large language model for long video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14313–14323, 2024. 6
- [22] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. Llava-pruner: Adaptive token reduction for efficient large multimodal models. *arXiv preprint arXiv:2403.15388*, 2024. 2
- [23] Karen Simonyan and Andrew Zisserman. Two-stream convolutional networks for action recognition in videos. *Advances in neural information processing systems*, 27, 2014. 12
- [24] Enxin Song, Wenhao Chai, Guan hong Wang, Yucheng Zhang, Haoyang Zhou, Feiyang Wu, Haozhe Chi, Xun Guo, Tian Ye, Yanting Zhang, et al. Moviechat: From dense token to sparse memory for long video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18221–18232, 2024. 6
- [25] Zhan Tong, Yibing Song, Jue Wang, and Limin Wang. Videomae: Masked autoencoders are data-efficient learners for self-supervised video pre-training. *Advances in neural information processing systems*, 35:10078–10093, 2022. 12
- [26] Du Tran, Lubomir Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. Learning spatiotemporal features with 3d convolutional networks. In *Proceedings of the IEEE international conference on computer vision*, pages 4489–4497, 2015. 12
- [27] Jue Wang, Wentao Zhu, Pichao Wang, Xiang Yu, Linda Liu, Mohamed Omar, and Raffay Hamid. Selective structured state-spaces for long-form video understanding. In *Proceed-*

ings of the *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6387–6397, 2023. [6](#)

- [28] Limin Wang, Yuanjun Xiong, Zhe Wang, Yu Qiao, Dahua Lin, Xiaoou Tang, and Luc Van Gool. Temporal segment networks: Towards good practices for deep action recognition. In *European conference on computer vision*, pages 20–36. Springer, 2016. [12](#)
- [29] Xidong Wang, Dingjie Song, Shunian Chen, Junyin Chen, Zhenyang Cai, Chen Zhang, Lichao Sun, and Benyou Wang. Longllava: Scaling multi-modal llms to 1000 images efficiently via a hybrid architecture. *arXiv preprint arXiv:2409.02889*, 2024. [6](#)
- [30] Shiwei Wu, Jia Chen, Kevin Qinghong Lin, Qimeng Wang, Yan Gao, Qianli Xu, Tong Xu, Yao Hu, Enhong Chen, and Mike Zheng Shou. Videollm-mod: Efficient video-language streaming with mixture-of-depths vision computation. *Advances in Neural Information Processing Systems*, 37:109922–109947, 2024. [2](#)
- [31] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. Visionzip: Longer is better but not necessary in vision language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 19792–19802, 2025. [1](#), [2](#), [4](#)
- [32] Sihan Yang, Runsen Xu, Chenhang Cui, Tai Wang, Dahua Lin, and Jiangmiao Pang. Vflowopt: A token pruning framework for llms with visual information flow-guided optimization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 23924–23934, 2025. [1](#), [2](#), [4](#), [5](#), [6](#)
- [33] Hang Zhang, Xin Li, and Lidong Bing. Video-llama: An instruction-tuned audio-visual language model for video understanding. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 543–553, 2023. [2](#), [6](#)
- [34] Peiyuan Zhang, Kaichen Zhang, Bo Li, Guangtao Zeng, Jingkang Yang, Yuanhan Zhang, Ziyue Wang, Haoran Tan, Chunyuan Li, and Ziwei Liu. Long context transfer from language to vision. *arXiv preprint arXiv:2406.16852*, 2024. [2](#), [4](#)

## 7. Complexity Analysis

**Notation.**  $T$ : frames;  $N$ : tokens per frame;  $k$ : intra-frame split factor;  $G$ : inter-frame retention ratio;  $T' = G \cdot T$ ;  $N' = N/k$ ;  $C$ : feature dimension (constant).

### 7.1. Vanilla attention cost

Standard self-attention over a flat sequence of  $T \times N$  visual tokens:

$$C_{\text{vanilla}} = \mathcal{O}((T \cdot N)^2) = \mathcal{O}(T^2 N^2). \quad (10)$$

### 7.2. VETO end-to-end cost

VETO introduces three operations before final attention.

**(1) Intra-frame compression** computes a  $N_{\text{src}} \times N_{\text{dst}}$  cosine similarity matrix per frame where  $N_{\text{src}} \approx N(1 - 1/k)$  and  $N_{\text{dst}} \approx N/k$ . Total over  $T$  frames:

$$C_{\text{intra}} = \mathcal{O}(T \cdot N^2/k). \quad (11)$$

**(2) Inter-frame similarity** mean-pools  $T$  spatially-compressed frames into  $C$ -dimensional vectors, then computes a  $T \times T$  cosine matrix:

$$C_{\text{inter}} = \mathcal{O}(T^2 \cdot C). \quad (12)$$

Since  $C$  is a fixed LLM hidden dimension while  $N$  scales with resolution, this term is negligible.

**(3) Final attention** over the compressed sequence of  $T' \times N'$  tokens:

$$C_{\text{attn}} = \mathcal{O}((T' \cdot N')^2) = \mathcal{O}\left(\frac{G^2 T^2 N^2}{k^2}\right). \quad (13)$$

Total VETO cost (dominant terms):

$$C_{\text{VETO}} = \mathcal{O}\left(\frac{T N^2}{k} + T^2 C + \frac{G^2 T^2 N^2}{k^2}\right). \quad (14)$$

### 7.3. Speedup ratio

The attention speedup factor is simply:

$$\text{Speedup}_{\text{attn}} = \frac{T^2 N^2}{G^2 T^2 N^2 / k^2} = \left(\frac{k}{G}\right)^2. \quad (15)$$

**At default settings** ( $k = 2$ ,  $G = 0.75$ ):

$$\text{Speedup}_{\text{attn}} = \left(\frac{2}{0.75}\right)^2 \approx 7.1 \times \quad (16)$$

reduction in attention FLOPs relative to vanilla.

### 7.4. Concrete example at VideoMME settings

VideoMME uses  $T = 64$  frames of  $N = 729$  tokens ( $27 \times 27$  ViT patch grid). Table 5 compares operation counts at these settings.

VETO’s compression overhead (intra + inter) totals  $\approx 0.7\%$  of vanilla attention—negligible. Net attention FLOPs reduction:  $\approx 7.1 \times$ .

Table 5. Operation counts at  $T = 64$ ,  $N = 729$ ,  $k = 2$ ,  $G = 0.75$ . Compression overhead is one to two orders of magnitude smaller than the attention saving it enables.

| Operation          | Seq. length | Rel. FLOPs         |
|--------------------|-------------|--------------------|
| Vanilla attention  | 46,656      | 1.000×             |
| VETO intra (total) | —           | 0.007×             |
| VETO inter         | —           | $\ll 0.001 \times$ |
| VETO attention     | 17,520      | <b>0.141×</b>      |

### 7.5. Single-axis methods cannot achieve the same reduction

**Intra-only** ( $k = 2$ , no temporal compression) yields  $T \times N/k = 64 \times 365 = 23,360$  tokens. Attention FLOPs:  $(23,360/46,656)^2 \approx 0.25 \times$ —a  $4 \times$  reduction.

**Inter-only** ( $G = 0.75$ , no spatial compression) yields  $T' \times N = 48 \times 729 = 34,992$  tokens. Attention FLOPs:  $(34,992/46,656)^2 \approx 0.56 \times$ —a  $1.8 \times$  reduction.

VETO’s dual-axis achieves  $7.1 \times \approx 4 \times 1.8$ —the two reductions compound because the two axes are independent.

Additionally, inter-only methods must compute the  $T \times T$  similarity matrix *on full- $N$  spatial vectors*, before any spatial compression has occurred. This early large-scale operation introduces a latency floor that the reduced final attention does not fully offset—which is precisely why Table 2a of the main paper shows Inter-only latency at  $1.16 \times$  relative to VETO, not below 1.00.

## 8. InternVL-2.5 on EgoSchema

Table 6 extends Table 1b of the main paper (LLaVA-OV-7B) to InternVL-2.5-1B on EgoSchema. VETO achieves a **23% latency reduction** with only a 0.7% accuracy drop—confirming cross-architecture efficiency. ToMe and HERMES are excluded due to limited InternVL-2.5 support.

Table 6. EgoSchema on InternVL-2.5-1B (V100 lat.). Accuracy (%) and normalized latency ( $\downarrow$  better).

| Method                   | Acc.        | Lat.        |
|--------------------------|-------------|-------------|
| <i>InternVL-2.5 (1B)</i> |             |             |
| Vanilla                  | <b>39.8</b> | 1.00        |
| <b>VETO (Ours)</b>       | 39.1        | <b>0.77</b> |

Combined with Table 1b (LLaVA-OV-7B: 60.2%/0.60 $\times$ ), VETO achieves consistent latency gains across two architecturally distinct families on this benchmark.

## 9. Standard Action Recognition: UCF-101

Table 7 evaluates the fully-supervised variant from the main paper on UCF-101—a short-clip benchmark with *less* tem-

poral redundancy than the long-form tasks in the main paper.

Table 7. UCF-101 action recognition. VETO uses the same frozen-backbone / linear-adaptor setup as the main paper’s supervised experiments.

| Method             | Acc. (%)    |
|--------------------|-------------|
| TS ConvNets [23]   | 88.0        |
| C3D [26]           | 82.3        |
| VideoMAE [25]      | 91.3        |
| TSN [28]           | 94.2        |
| TSM [15]           | <b>95.9</b> |
| <b>VETO (Ours)</b> | <b>94.4</b> |

**Training setup.** 100 frames per clip,  $k = 2$ ,  $G_{\text{ratio}} = 0.20$  (aggressive temporal compression reflects short clips having less inter-frame redundancy than long instructional video), 20 epochs, batch size 4.

**Interpretation.** VETO (94.4%) matches TSN (94.2%) and falls within 1.5% of TSM (95.9%), a specialist two-stream architecture optimized specifically for action recognition. The result demonstrates that the uniform-anchor spatial matching preserves discriminative features even at  $k = 2$  compression, and that the dual-axis design generalizes to temporal scales well outside its primary design target.

## 10. Extended Ablation Discussion

This section provides deeper mechanistic analysis of the ablation results in Table 2 of the main paper.

**Why does Intra-only inflate latency?** Intra-only compresses each frame from  $N$  to  $N/k$  tokens but applies no temporal compression. At default settings ( $T = 64$ ,  $k = 2$ ): final sequence has  $64 \times 365 = 23,360$  tokens. VETO’s final sequence has  $48 \times 365 = 17,520$  tokens. The Intra-only sequence is  $1.33\times$  longer, producing the  $1.10\times$ – $1.14\times$  latency overhead seen in Table 2a.

**Why does Inter-only inflate latency more than Intra-only?** Inter-only retains all  $N = 729$  spatial tokens per frame, yielding  $48 \times 729 = 34,992$  tokens—twice as many as VETO. Worse, the inter-frame similarity step runs on full- $N$  feature vectors, adding expensive overhead *before* any token reduction is realized. Both effects compound to produce  $1.16\times$  latency—the highest of all four configurations.

**Why does Inter-first still incur overhead despite reaching the same final token count as Intra-first?** Both Inter-first and Intra-first end with  $T' \times N/k$  tokens. The difference is entirely in the inter-frame similarity step:

- **Intra-first:** inter similarity computed on  $N/k = 365$ -dim mean-pooled vectors  $\Rightarrow$  cheaper.

- **Inter-first:** inter similarity computed on  $N = 729$ -dim mean-pooled vectors  $\Rightarrow 2\times$  more expensive.

This single upstream difference accounts for the  $1.07\times$ – $1.09\times$  overhead of Inter-first. The lesson: it is not enough to achieve the same *final* compression ratio—the *order* of operations determines the overhead incurred along the way.

**Spatial vs. temporal sensitivity: the asymmetry explained.** The intra-frame ablation (Table 2b) shows  $<0.6\%$  VideoMME degradation across  $k \in \{2, 3, 4, 5\}$  (retaining 20–50% of per-frame tokens). The inter-frame ablation (Table 2c) shows 6.8% degradation at  $G_{\text{ratio}} = 0.125$  (retaining 12.5% of frames).

The structural explanation: a frame’s  $N$  tokens encode a static scene where neighboring patches are often semantically redundant (uniform backgrounds, repeated textures). The uniform modulo- $k$  partition guarantees every spatial region contributes one anchor, so merging 50–80% of tokens is safe.

Temporal frames, by contrast, encode a causal sequence of events. Each discarded frame may contain a unique transition or action state that cannot be inferred from neighbors. This is why degradation becomes non-linear below  $G_{\text{ratio}} = 0.50$ : the model begins discarding irreplaceable causal information.

The recommended operating point ( $k = 2$ ,  $G_{\text{ratio}} = 0.75$ ) sits in the robust regime of both axes, validated empirically by Table 2b–c.